DAA

1):0/1 Knapsack

0/1 Knapsack is a dynamic programming problem where each item is either included or excluded to maximize total profit without exceeding the knapsack capacity.

2 Give Recurrence Relation (VERY IMPORTANT ⭐)

$$dp[i][w] = \begin{cases} \max(val[i] + dp[i-1][w - wt[i]],\ dp[i-1][w]) & \text{if } wt[i] \le w \\ dp[i-1][w] & \text{if } wt[i] > w \end{cases}$$

```
Algorithm Knapsack01(wt, val, n, W)

1. Create a 2D array dp[n+1][W+1]

2. For i ← 0 to n:
       dp[i][0] ← 0

3. For w ← 0 to W:
       dp[0][w] ← 0

4. For i ← 1 to n:
       For w ← 1 to W:

           If wt[i] ≤ w then
               dp[i][w] ← max(
                               val[i] + dp[i-1][w - wt[i]],
                               dp[i-1][w]
                              )
           Else
               dp[i][w] ← dp[i-1][w]

5. Return dp[n][W]

End Algorithm
```

⍰ Time Complexity: **O(nW)**

⍰ Space Complexity: **O(nW)**

->Numerical me u have to create a 2d matrix of order(n+1)(w+1) and for first row and first column make it zero(initialization in dp). fill the table using the recurrence realtion.

2) Coin Change Problem:

The Coin Change problem aims to determine the minimum number of coins required to make a given amount using unlimited supply of given coin denominations.

State Definition

Let

$$dp[i][j]$$

denote the minimum number of coins required to make amount **j** using the first **i** types of coins.

Recurrence Relation

$$dp[i][j] = \begin{cases} \min(1 + dp[i][j - coin_i],\ dp[i-1][j]) & \text{if } coin_i \le j \\ dp[i-1][j] & \text{if } coin_i > j \end{cases}$$

Explanation:

- $1 + dp[i][j - coin_i]$ → include current coin
- $dp[i-1][j]$ → exclude current coin

Base Cases

$$dp[i][0] = 0 \quad \text{for all } i$$

$$dp[0][j] = \infty \quad \text{for } j > 0$$

```
Algorithm CoinChange2D(coins, n, Amount)

1. Create a 2D array dp[0…n][0…Amount]

2. For i ← 0 to n do
       dp[i][0] ← 0

3. For j ← 1 to Amount do
       dp[0][j] ← ∞

4. For i ← 1 to n do
       For j ← 1 to Amount do

           If coins[i] ≤ j then
               dp[i][j] ← min(1 + dp[i][j – coins[i]],
                                dp[i–1][j])
           Else
               dp[i][j] ← dp[i–1][j]

5. If dp[n][Amount] = ∞ then
       return "Not Possible"
   else
       return dp[n][Amount]

End Algorithm
```

Time and Space Complexity

- **Time Complexity:** $O(n \times Amount)$
- **Space Complexity:** $O(n \times Amount)$

3) **Multistage Graphs**

Introduction

A multistage graph is a directed acyclic graph (DAG) in which the vertices are divided into stages. Edges exist only from a vertex in stage $i$to a vertex in stage $i+1$. The objective is typically to find the minimum cost path from the source (first stage) to the sink (last stage). Dynamic programming is used to solve this efficiently.

Properties

- The graph is divided into **k stages**.
- There is a **source** in stage 1 and a **sink** in stage k.
- Edges go only from stage $i$to stage $i+1$.
- The graph is **acyclic**.
- Used for shortest path problems.

State Definition

Let

$$cost[i] = \text{minimum cost from vertex } i \text{ to the sink}$$

Recurrence Relation

$$cost[i] = \begin{cases} 0 & \text{if } i \text{ is sink} \\ \min\limits_{(i,j)\in E} \{w(i,j) + cost[j]\} & \text{otherwise} \end{cases}$$

where $w(i,j)$ is the weight of edge from i to j.

Backward Approach:

```
Algorithm MultistageGraph(G, n)

1. cost[n] ← 0                    // n is sink

2. For i ← n-1 down to 1 do
       cost[i] ← ∞

       For each vertex j such that (i, j) ∈ E do
            If w(i, j) + cost[j] < cost[i] then
                  cost[i] ← w(i, j) + cost[j]
                  next[i] ← j

3. Return cost[1]

End Algorithm
```

Path Reconstruction

To print the path:

i ← 1
print i

While i ≠ n do
    i ← next[i]
    print i

Time and Space Complexity

- **Time Complexity:** $O(E)$
- **Space Complexity:** $O(V)$

4) **All-Pairs Shortest Paths (Floyd–Warshall Algorithm)**

Introduction

The Floyd–Warshall algorithm is a dynamic programming technique used to find the shortest paths between every pair of vertices in a weighted graph. It works for both directed and undirected graphs and can handle negative edge weights (but not negative cycles).

Idea

The algorithm progressively improves the shortest path estimates by allowing intermediate vertices one by one.

Let

$$D^{(k)}[i][j]$$

denote the shortest distance from vertex $i$to vertex $j$using only vertices $1,2, \dots, k$as intermediate vertices.

Recurrence Relation

$$D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)$$

Explanation:

- Either the shortest path from i to j does **not** go through k, or
- It goes through k.

```
Algorithm FloydWarshall(W, n)

1. Let D be an n × n matrix
2. D ← W                        // initialize with weight matrix

3. For k ← 1 to n do
4.     For i ← 1 to n do
5.         For j ← 1 to n do
6.             If D[i][k] + D[k][j] < D[i][j] then
7.                 D[i][j] ← D[i][k] + D[k][j]

8. Return D

End Algorithm
```

Time and Space Complexity

- **Time Complexity:** $O(n^3)$
- **Space Complexity:** $O(n^2)$

5) **Single Source Shortest Path (Bellman–Ford Algorithm)**

Introduction

The Bellman–Ford algorithm is used to compute the shortest path from a single source vertex to all other vertices in a weighted graph. It can handle graphs with negative edge weights and also detects the presence of negative weight cycles.

Idea

The algorithm repeatedly relaxes all edges.
After at most $V - 1$iterations (where $V$is the number of vertices), the shortest paths are guaranteed to be found if no negative cycle exists.

Relaxation Principle

For an edge (u, v) with weight $w(u, v)$:

$$\text{if } dist[u] + w(u, v) < dist[v] \Rightarrow dist[v] = dist[u] + w(u, v)$$

```
Algorithm BellmanFord(G, V, E, source)

1. For each vertex v in V do
       dist[v] ← ∞

2. dist[source] ← 0

3. For i ← 1 to V-1 do
       For each edge (u, v) with weight w in E do
            If dist[u] + w < dist[v] then
                 dist[v] ← dist[u] + w
                 parent[v] ← u

4. // Negative cycle detection
   For each edge (u, v) with weight w in E do
        If dist[u] + w < dist[v] then
             report "Negative weight cycle exists"

5. Return dist[ ]

End Algorithm
```

Time and Space Complexity

- **Time Complexity:** $O(VE)$
- **Space Complexity:** $O(V)$

```
Algorithm PrintPath(parent, s, t)

1. If t = s then
       print s
       return

2. PrintPath(parent, s, parent[t])
3. print t
```